

Where the Seconds Go: Bounding Mixture-of-Experts Decode from Host Memory, and Measuring What Foresight Is Worth

Harshith Kantamneni

Independent Researcher

harshithkantamneni@users.noreply.github.com

Abstract—A Mixture-of-Experts model too large for one consumer GPU keeps most of its experts in host memory, and decoding one token at a time then waits on host reads. We bound how fast such decoding can be on a given machine, measure where the remaining time goes, and price knowledge of the future routing. The bound is the fewest host reads any expert cache of a given size can make (Belady’s MIN with bypass) at the machine’s probed read rate. Our llama.cpp expert cache, faster than two other systems at most settings, reaches 31–54% of the bound’s speed on consumer machines at the smallest cache we test. Oracles built into the engine split the remaining time by what is cached, how it is read and when. The engine’s fetch table, which copies misses into cache slots the policy did not choose, hides the value of caching MIN’s set: with it, the set pays only if each admitted expert is also read once; without it, on three machines, the set alone closes 20% of the gap and both changes together 44%. Even the read-ahead oracle leaves about half of the gap. On five fast-link machines, policies without foresight recover at most 6% of its gain. In trace replay, half of foresight’s value needs the routing of about 0.65 cache-fulls of distinct experts ahead. Registered predictions were committed before their machines were rented; the consumer range and the replay result are post hoc.

Index Terms—Caching, foresight, mixture of experts, offloading, performance bounds

I. INTRODUCTION

Open-weight Mixture-of-Experts (MoE) models now carry 30–120 B parameters and activate a few billion per token; a desktop GPU has 24–32 GB. Running such a model for one user therefore means keeping most experts in host DRAM. Each decoded token reads its routed experts from GPU memory if they are resident, and otherwise from host memory: either the CPU runs the expert, or the expert is copied over PCIe into a GPU slot [1], [2].

Many systems build on this: GPU expert caches [3]–[5], CPU execution of misses [6]–[8], a per-miss choice between the two [9], [10] and expert prediction [11]–[13]. Each reports a speed-up over a baseline of its choosing. None says how fast the same machine could decode the model, or which part of the remaining time a better design could remove. Without that, a builder cannot tell whether to work on what is cached, how it is read, when it is read, or how far ahead the routing must be predicted.

Appendices A–N and the tables, figures and equations numbered S1, S2, ... are in the supplementary material available online.

We answer these questions for batch-1 decode of gpt-oss-120b [14] and Qwen3-30B-A3B [15] on rented RTX 5090 machines. We derive a bound, build oracles into a real engine that remove one part of the gap at a time, and run them on many machines, with every prediction committed before its machine started. Table I lists the answers and the evidence behind each.

The headroom is large (Section III). MIN’s host reads at the machine’s probed rate bound the time per token of any system with C slots per layer, and that read time is nearly reachable in a microbenchmark. Our cache, which leads FreeToken and llama.cpp at most configurations, reaches 31–54% of the bound’s speed at the smallest gpt-oss budget on consumer machines (a scope set post hoc). Scored like-for-like on datasheet rates (post hoc), ours reaches a median 27% and published systems 13.6%.

What is cached pays if the read path keeps it (Section IV). Our engine’s deployed read path also copies some misses into cache slots the policy did not choose. With these in-step fetches on, MIN’s set gains nothing unless each admitted expert is also read once instead of twice; then the two changes close 39% of the gap on fast-link machines at the smallest gpt-oss budget. With them off, on three machines of a registered test, MIN’s set alone closes 20% of the gap, reading the deployed set once 10%, and both together 44%: the set pays by itself, and more when read once.

Without foresight, little of it is reachable (Section VI). Four variants built from the engine’s admission margin and layer-ahead prediction recover at most 6% of the read-ahead oracle’s gain on five fast-link machines registered for the test: they read as many experts as the deployed cache.

The machine decides how foresight should be spent (Sections V and VII). Admit as few experts as optimality allows, keep them, and read them on the path the link-to-CPU ratio favours; where the link is fast, at the smallest budget, reading each admission once pays most. A trend in that ratio, fitted on RTX 5090s, predicted the oracles’ speed on five RTX 4090s within 7%; applied after the fact to the five new RTX 5090s, it missed by up to 18%.

Foresight has a price (Section VIII). In replay of 9 models’ routing traces (post hoc), half of its value needs the routing of about 0.65 C distinct experts per layer ahead; exact routing of the next token recovers at most a third of it on our workload.

TABLE I

OUR CLAIMS, THE THRESHOLD REGISTERED FOR EACH BEFORE ITS MACHINES STARTED, AND THE RESULT (TABLE S3 POINTS TO EACH CLAIM’S DATA AND CLAUSES). *Derived*: FOLLOWS FROM THE DEFINITIONS; *post hoc*: FOUND IN THE DATA IT DESCRIBES; *loose*: A THRESHOLD THE EARLIER DATA MADE EASY TO MEET; *held*: THE WHOLE 95% INTERVAL MEETS THE THRESHOLD; *held (point)*: ONLY THE POINT ESTIMATE DOES. *Mach.*: MACHINES THAT PASSED EVERY CHECK (FOR THE 2×2, THE PANEL’S MAIN JOB AND THE NEW MACHINES).

Claim	Registered threshold	Result	Mach.	§
MIN’s reads at the probed rate bound any exact-routing cache with C slots per layer	derived, relative to the probe	by Eq. (S1)’s accounting the engine’s reads imply a rate $>5\%$ above the probe’s on 12 of 84 launch-budgets	–	III
The bound’s read time is nearly reachable	$\geq 50\%$ of it per layer (loose)	86–96%, held (point); 5 of the job’s 33 clauses failed	3	III
Our cache is faster than FreeToken and llama.cpp at most configurations	on B: faster than FreeToken (interval > 1), $\geq 1.8\times$ llama.cpp ($\geq 2\times$ at Qwen3 12.5%); on S: within ± 0.06 of B’s ratios	held on B; on S, five of six ratios fell outside ± 0.06 of B’s; faster at 11 of 12 in all	B, S	III
On the deployed read path, MIN’s set pays only together with one read (gpt-oss 11%, fast links, greedy schedule)	interaction > 0 on every machine; MIN-1R’s mean share of the gap 0.25–0.55 (loose)	positive on 9 of the panel’s 10 main-job machines (failed), 5 of 5 new ones (held (point); fast-link scope set in between); share 35%	10 + 5	IV
Control: issuing the two-read arm’s copies a step earlier, to separate when a copy lands from the second read	misses $\leq 0.95\times$ Few-2R’s; slowdown ≤ 0.01 ; per machine and budget	at 11%: misses 0.97–0.98 $\times$, 0.03–0.05 slower; failed on 5 and 7 of 8 machine-budgets; so the control is uninformative	4	IV
With the in-step fetches off, MIN’s set pays alone, and more with one read	per machine, CPU-only path: MIN-2R $\geq 1.05\times$ (11%) and $\geq 1.15\times$ (25%) the deployed cache (loose: below job 113’s values); Dep-1R 0.95–1.10 $\times$; interaction > 0	MIN-2R 1.10–1.12 $\times$ and 1.28–1.36 $\times$, Dep-1R 1.03–1.07 $\times$, interaction +0.4–3.2 ms (held); two checks of the fetches’ removal failed where the table was all zero	3	IV
Without the fetches, the fewest-admission set pays read twice	per machine: Few-2R $\geq 1.10\times$, MIN-2R $\geq 1.03\times$ a deployed cache without them; one read ≥ 0.05 more; deployed 0.80–1.00 $\times$	Few-2R 1.23–1.26 $\times$, MIN-2R 1.09–1.12 $\times$ (held); one read +0.02–0.16, deployed 0.98–1.07 $\times$ (each failed on the Core Ultra 9 285K, at 11%)	2	IV
Policies without foresight recover little of the oracle’s gain	speed $\leq 1.10\times$; capture $\leq 35\%$ (loose), per machine	best 1.007–1.024 $\times$, capture at most 6% (held)	5	VI
MIN’s fewest-admission set read once gains on stable machines	> 1 on every machine	1.26 $\times$ (geometric mean) on all 15 stable machines, held on the 7 registered; of the four that ran unsteadily or failed a validity check, lost on two (one registered, failed; one excluded by its job’s validity rule)	15	V
The link-to-CPU ratio predicts the oracles’ speed on a second card	within 0.10 in log of the RTX 5090 trend (trend fitted post hoc)	within 7% at ratios 0.37–0.77 (held); new fast-link RTX 5090s off by up to 18% (post hoc)	5	VII
Our cache reaches 31–54% of the bound on consumer machines (gpt-oss 11%)	post hoc; consumer scope set after the data	31–54% on 25 consumer machines; 12–46% on server ones	25	III
Published systems reach a median 13.6% of their bound	post hoc	their papers’ hardware at datasheet rates	–	III
Half of foresight’s value needs $\approx 0.65 C$ distinct experts ahead	post hoc	on 9 models’ traces	–	VIII

Section IX turns these results into rules for system builders; Appendices D and N give every prediction, including the failures, and the artifact.

II. SETTING

Models and workload: gpt-oss-120b has 36 MoE layers of 128 experts, top-4, in MXFP4 (13.25 MB per expert). Qwen3-30B-A3B has 48 layers of 128 experts, top-8, in BF16 (9.44 MB per expert). Prompts are AIME-25 problems, and the cache carries over between problems, as in one user’s session. The system comparison decodes 256 tokens of each of the 30 problems through each system’s server. The engine experiments replay the model’s own greedy output, teacher-forced, so that every configuration sees the same routing. A *budget* is the share C/E of each layer’s experts that fit in GPU slots: gpt-oss 11% and 25% are $C=14$ and $C=32$ of 128. The two smallest budgets of each model are *host-bound*: host reads, not the GPU, limit their speed (Section III gives the exceptions). Most results are at gpt-oss 11% and 25%.

The engine: Our cache is a 3,500-line patch to llama.cpp [16]. Each MoE layer has C GPU slots. A *miss* is read from host memory in one of two ways: CPU helper threads run it, or it is *fetched* (copied into a slot over PCIe and run on the GPU in the same step). A per-machine table, computed from the machine’s bandwidth probe, sets how many of a layer’s misses are fetched. The *deployed* policy admits a miss into a slot when its decayed request count beats the lowest resident’s by a margin $\kappa=1$. The CPU serves the admitted miss and a background copy then fills its slot, so the expert is read from host memory twice (Fig. 1); the copy is published at the start of the second step after the one that admitted it.

Terms: *MIN* is Belady’s optimum with bypass [17], [18]: it *admits* a miss only if some resident is needed later than the miss, which gives the most hits possible with C slots [19], [20]. Many schedules reach that optimum. The *greedy* one admits whenever it may; the *fewest-admission* one admits as rarely as any of them. An admitted expert is read *once* (1R), fetched into its slot in the step that needs it, or *twice* (2R),

TABLE II
THE NAMES THIS PAPER USES: CONFIGURATIONS (LEFT), MACHINE SETS AND YARDSTICKS (RIGHT).

Admitted set	read twice	read once	read ahead	<i>Machine sets</i> (RTX 5090 unless noted)	
<i>No foresight</i>				panel	the 15 machines that ran every state of Table IV before the new machines; 10 ran its main job
deployed: decayed count beats the lowest resident's by $\kappa=1$ the same, larger margin ($\kappa=3$ at 11%, 2 at 25%) the same, plus a layer-ahead copy ^a	deployed cache Margin LA	Dep-1R – LA-1R, LA-1R-margin ^b admit every miss	– – –	new machines	5 rented for the registered test of Section IV (ratio ≥ 0.5)
every miss	–	admit every miss	–	O3–O5	three machines that ran every budget (Section V)
<i>Oracles</i> (the future routing, from a trace recorded on the same machine)	MIN-2R	MIN-1R	read-ahead oracle ^c	hosts B, S	the system comparison (Table III)
MIN with bypass, greedy schedule	MIN-2R	MIN-1R	read-ahead oracle ^c	RTX 4090s	5 machines of the second-card tests (Section VII)
MIN with bypass, fewest admissions	Few-2R ^d	Few-1R	–	<i>Yardsticks</i> (T : time per token on one machine)	
Belady's rule, no bypass	Belady-2R	–	Belady-1R	speed	T_{deployed}/T ($\times$), the default
				share of the bound	T_{bound}/T , the bound being Eq. (1)
				share of the gap	$(T_{\text{deployed}} - T)/(T_{\text{deployed}} - T_{\text{bound}})$
				capture	speed gain over the read-ahead oracle's gain
				value of	share of MIN's host-read saving over admitting
				foresight	every miss

Read twice: the CPU serves the miss, then a background copy fills the slot; *read once*: fetched into the slot in the step. ^a A copy, one layer ahead, of the non-resident expert that the next layer's router, applied to this layer's input, ranks highest; an admission it predicted is read once. ^b LA-1R with Margin's margin. ^c MIN's admissions copied three steps ahead where the next use allows. ^d Few-2R-early: each copy issued one step earlier, so that it lands for the next use. *CPU only*: a configuration with the in-step fetches off, every miss served by the CPU (Section IV).

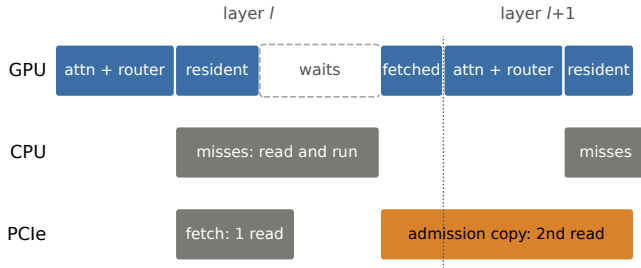


Fig. 1. One decode step of the deployed cache, schematically. Blue: the GPU's own kernels. Grey: host reads of the misses, which the step waits for. Orange: the background copy that fills an admitted expert's slot, a second read of the same expert.

served by the CPU in that step and copied in the background. A copy issued a few steps before the expert's use is read ahead. *Oracles* read the future routing from a trace recorded on the same machine. Configurations are named by the set admitted and the number of reads: MIN-1R is MIN's greedy set read once, MIN-2R the same set read twice, Dep-1R the deployed set read once, and Few-1R and Few-2R MIN's fewest-admission set. Table II lists the configurations, machine sets and yardsticks (the window policies of Section VIII are in Appendix G), and Appendix C maps the codes of the appendix tables to these names.

Machines: Every machine is a rented host with one RTX 5090 (an RTX 4090 in Section VII). Each runs our bandwidth probe before any model run. It reads host memory from CPU threads, at the number of helper threads the engine uses (B_c), and over PCIe into the GPU, at the rate the engine's copies use (B_p); B_{host} is its highest reading by any method. The *link-to-CPU ratio* is B_p/B_c . We call a link *fast* when the ratio is at

least 0.5 and *slow* otherwise; the term compares the link with the CPU, not with other links, and we drew the 0.5 line after the panel's data, before the registered tests of Sections IV, VI and VII. Ryzen, Core and Threadripper parts, high-end desktop ones included, are *consumer* processors; EPYC, Xeon and engineering-sample parts are *server* processors.

Statistics and registration: Time per token is averaged over problems, and speed ratios are ratios of mean times. Machines differ far more than problems (two panel machines with the same CPU model differ by up to 29% in the deployed cache's time), so the machine is the unit. Claims are about orderings that hold on every machine of a stated class, and summaries over machines carry 95% t -intervals over machines, since with a handful of machines a percentile bootstrap over machines is too narrow [21]; comparisons on one machine carry 95% intervals over problems (a paired bootstrap). A *launch* is one rental; in the later experiments each configuration ran in two or three processes, *rounds*, per launch (at 25% in the registered tests, one). A machine *ran stably* when its outputs matched the reference and its processes' times agreed within 2% (Appendix D). Intervals over problems leave out rounds and rentals; a speed ratio moved by a median 0.2% between rounds and 0.5% between rentals, and Appendix D rescores the registered clauses with both. Every experiment's predictions were committed to a public branch before its machine started; Appendix D scores all of them, and Table I marks the thresholds that earlier data made easy to meet.

III. HOW FAST COULD IT BE?

Consider any system that executes the model's exact routing with at most C experts of each layer resident on the GPU. Per token, each of the Lk routed experts is served by a resident copy or read from host memory. Let R^* be MIN's host reads

per token, summed over layers, on the routing trace the engine runs (Appendix N); no policy, even one that knows the future, reads fewer. If m experts per token run on the CPU, host memory supplies at least $\max(R^*, m)S$ bytes for experts of S bytes, and the GPU reads the dense weights D plus $(Lk - m)S$. With everything overlapped,

$$\bar{T} \geq \min_{0 \leq m \leq Lk} \max\left(\frac{D + (Lk - m)S}{B_{\text{gpu}}}, \frac{\max(R^*, m)S}{B_{\text{host}}}\right). \quad (1)$$

At a host-bound budget this is MIN's read time, R^*S/B_{host} . The bound assumes exact routing, whole experts, at most C residents per layer and one token per step. It drops all latencies, lets the two read paths share B_{host} , and is relative to the probe: a reader faster than the probe could beat it (Section XI).

The demand bound: reads that wait for the router: A system that reads an expert only after its layer's router has chosen it cannot overlap that read with the GPU's work: layer l 's misses are known only after its attention and router, and layer $l+1$ starts only after layer l 's experts. Its time is at least the GPU's non-expert work plus the reads, in series:

$$\bar{T}_{\text{demand}} \geq T_{\text{GPU}} + \frac{R^*S}{B_{\text{host}}}, \quad (2)$$

where T_{GPU} is the GPU's attention, dense layers, norms and routers per token: 2.9 ms for gpt-oss, the smallest of the 14 Nsight profiles of our engine we use; a system with faster non-expert kernels has a lower bound. Every system we compare reads on demand; only reads issued before routing can go below Eq. (2).

The ordered bound: reads foresight cannot free: Foresight does not free every read. An expert run on the CPU needs its layer's attention output, and the next layer's attention needs its result, so CPU reads sit in series with the GPU's non-expert work; only copies over the link can be issued ahead. This assumes that the CPU reads an expert's weights from memory when it runs it; a read-ahead into the CPU's cache would relax it. With a share f of MIN's reads served by the CPU at the probe's highest CPU reading $\hat{B}_c$ and the rest copied at its highest link reading $\hat{B}_p$,

$$\bar{T} \geq \min_{0 \leq f \leq 1} \max\left(\frac{(1-f)R^*S}{\hat{B}_p}, T_{\text{GPU}} + \frac{fR^*S}{\hat{B}_c}\right), \quad (3)$$

and Eq. (1) holds as well, so the bound is the larger of the two. On the median consumer machine at gpt-oss 11%, Eq. (3) equals Eq. (1) (ratio 1.00); where the link is slow, it is up to 1.56 times as high.

The read time is nearly reachable: A microbenchmark replays MIN's per-layer reads of gpt-oss on 3 machines without the model: CPU threads read part of each layer's experts while the copy engine copies the rest. It reaches 86–96% of the bound's read time (Table S25; registered: at least 50%, a loose threshold). The GPU term of Eq. (1) is looser, but at datasheet rates it binds only at the largest budgets and at gpt-oss 25% on the machines with the fastest host memory (Table S24); at the GPU's measured batch-1 rate it also binds at gpt-oss 25% on both hosts of Table III and at Qwen3 25% on host B.

Where systems stand: Table III places three systems against the bounds on two machines. Our cache leads Free-Token [9] at 11 of 12 configurations and stock llama.cpp at all of them (Appendix J). Across 25 machines with consumer processors at gpt-oss 11%, it runs at 31–54% of the speed of Eq. (1), the bound we use throughout (40–54% of the ordered bound, Eq. (3)); the 5 machines added in Section IV fall at 41–48%. The top of that range is probably too high: on that machine the engine's reads imply a rate above the probe's best reading (Table S23).

Published systems (post hoc): Scored against the same bound on their papers' own hardware at datasheet rates, 20 published batch-1 measurements reach a median 13.6%; ours, scored the same way, a median 27% (Appendix M, which gives the caveats).

IV. WHERE THE SECONDS GO

The gap is the deployed cache's time beyond Eq. (1). Oracles in the engine remove three parts of it: what is cached (MIN's set instead of the deployed one), how it is read (each admission fetched once in the step, not served by the CPU and copied in the background), and when (copies issued a few steps early). The deployed read path also has *in-step fetches*: a per-machine fetch table copies some misses into slots the policy did not choose. We measure the 2×2 on that path, on the 15 panel machines and 5 machines rented for a registered test, and on a CPU-only path without in-step fetches, on 3 machines of a second registered test (Fig. 2 and Tables IV and V).

The in-step fetches displace the set the policy chose: On the two machines whose probe gave a nonzero fetch table, MIN-2R missed 48.6–50.8 experts per token at gpt-oss 11% with the in-step fetches and 43.5 without them: the table copies misses MIN bypasses into slots MIN keeps. Most of Dep-1R's in-step fetches were the table's too, 23.4–32.0 per token against its 8.2 admissions. The third machine's probe gave an all-zero table (no in-step fetches); MIN-1R's forced plans replace the table, so it holds MIN's set on either path.

Without them, MIN's set pays alone, and more with one read: On the CPU-only path at 11%, MIN's set read twice ran 1.10–1.12 $\times$ the deployed cache and closed a mean 20% of the gap; the deployed set read once ran 1.03–1.07 $\times$ and closed 10%; MIN-1R closed 44%. Together the two still pay more than apart, by 0.4–3.2 ms per token, its interval above zero on all 3 machines (registered: positive): MIN's set admits 20.4 experts per token against the deployed set's 8.2, so reading each admission once saves more for it. The excess is largest on the two Ryzen 9 5950X machines (Table V). At 25%, MIN's set alone closed 37% and the excess fell to about zero (–0.1 to 0.1 ms).

On the deployed path, MIN's set pays only together with one read: There, MIN-2R closes 0% of the gap at 11%, Dep-1R 0%, and MIN-1R 33% on the panel (39% on its 13 fast-link machines) and 35% on the new machines. We registered that this interaction would be positive on every machine; it was, on 9 of the 10 machines of the panel's main job and on all 5 new ones. On the two machines of the second test with a nonzero

TABLE III

DECODE SPEED AT EQUAL GPU EXPERT MEMORY ON TWO RTX 5090 HOSTS (30 AIME-25 PROBLEMS, THE FIRST 256 GREEDY TOKENS, ONE SESSION).

Model	Experts on GPU	llama.cpp (tok/s)	FreeToken (tok/s)	Ours (tok/s)	Ours $\div$ FreeToken (95% CI)	Ours $\div$ llama.cpp	Bound, datasheet tok/s	GPU ours	Bound, measured tok/s	GPU ours
Host B: Ryzen 9 9950X3D, card memory 17001 MHz; CPU 72, link 53, together 78, highest 88 GB/s (the bound's rate)										
gpt-oss	11%	34.9	54.0 ^h	69.9	1.29 [1.28, 1.31]	2.00 $\times$	172	41%	172	41%
gpt-oss	25%	40.0	85.6 ^h	109.2	1.27 [1.25, 1.29]	2.73 $\times$	430	25%	279	39%
gpt-oss	40%	47.7	132.2 ^o	152.5	1.15 [1.13, 1.17]	3.20 $\times$	519	29%	279	55%
Qwen3	12.5%	19.5	38.7 ^h	40.0	1.03 [1.02, 1.04]	2.05 $\times$	96	42%	96	42%
Qwen3	25%	22.1	54.7 ^h	63.1	1.15 [1.13, 1.17] [†]	2.86 $\times$	214	30%	192	33%
Qwen3	43.75%	28.4	103.1 ^o	108.2	1.05 [1.04, 1.06]	3.81 $\times$	308	35%	192	56%
Host S: Ryzen 9 9950X, stock clock, card 14001 MHz; CPU 56, link 53, together 66, highest 71 GB/s; FreeToken backends as on B										
gpt-oss	11%	28.9	48.0 ^h	57.9	1.21 [1.19, 1.22]	2.00 $\times$	140	41%	140	41%
gpt-oss	25%	33.6	78.7 ^h	94.1	1.20 [1.16, 1.22]	2.80 $\times$	351	27%	275	34%
gpt-oss	40%	40.1	122.2 ^o	133.6	1.09 [1.08, 1.11]	3.34 $\times$	514	26%	275	49%
Qwen3	12.5%	16.1	32.7 ^h	33.5	1.03 [1.02, 1.04]	2.08 $\times$	78	43%	78	43%
Qwen3	25%	18.5	51.4 ^h	53.9	1.05 [1.04, 1.06]	2.91 $\times$	174	31%	174	31%
Qwen3	43.75%	23.8	98.3 ^o	95.8	0.97 [0.96, 0.99]	4.02 $\times$	305	31%	189	51%

Ratios are of mean speeds; the ratio over FreeToken is paired by problem within one launch, with a 95% bootstrap interval over problems. Host B's rows come from three rentals of the same machine (the same GPU), so its ratios over llama.cpp compare launches. Our cache and FreeToken ran twice per cell and the second launch is shown (on host B both launches ran ours first); llama.cpp and the [†] cell ran once. Each system decodes its own greedy text. Our cache uses each machine's fetch table; FreeToken uses its faster backend per budget (^h hybrid, ^o offload), picked on host B on a separate launch. gpt-oss: gpt-oss-120b; Qwen3: Qwen3-30B-A3B. *Bound*: Eq. (1) on that machine (MIN's reads, the host's highest probed rate) with the GPU at its datasheet rate, and at the rate all-in-VRAM decode measures at batch size 1 (52% of datasheet for gpt-oss, 61% for Qwen3; Table S24 varies the other assumptions; Appendix J gives the pooled bounds and all-in-VRAM speeds). [†] Selection launch only, which favours FreeToken.

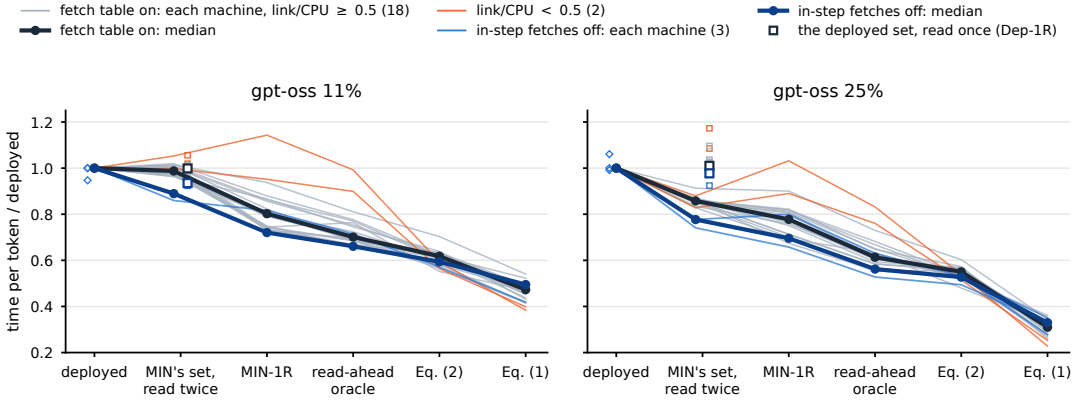


Fig. 2. Where the seconds go, measured: each state's time per token relative to the deployed cache on the same machine (lower is faster) as what is cached changes (MIN's set, read twice; squares: the deployed set, read once), then both, then when, with the two bounds. One line per machine. Grey (orange: link-to-CPU ratio below 0.5): the deployed read path, where the second state is MIN's schedule with the fetch table, not its set held. Blue: in-step fetches off (diamonds: the deployed cache without them). Thick: medians over fast-link machines.

table, most of it was the table's: it fell from 31–52% of the gap to 3–21% without the in-step fetches; on the machine without a table it was 21% either way.

What is left: The read-ahead oracle closes another 15% at 11%, and about half the gap remains (Table IV). On fast links two named parts account for it (at 25% they overshoot slightly). The oracle still reads its remaining misses after their router runs, so the GPU's non-expert time stays in series with them; and it reads more than MIN, because its early copies evict sooner. Removing both would take every read issued before routing at MIN's count, and only copies over the link can be issued early (Eq. (3)). On slow links the two parts fall short: at 11% a residual of 35–47% of the gap stays unexplained on

the two slow-link panel machines, and 31–36% on the three slow-link RTX 4090s of Section VII; measuring against Eq. (3) does not remove it (Appendix L).

V. HOW FORESIGHT MUST BE SPENT

This section measures each way of spending foresight as a speed relative to the deployed cache on the same machine. A running example: on a Ryzen 9 9950X at gpt-oss 11%, Eq. (1) gives 10.0 ms per token and Eq. (2) 13.0. The deployed cache takes 20.6 ms, MIN-1R 15.2, and the read-ahead oracle 13.8.

Reading each admitted expert once pays most: MIN-1R is faster than the deployed cache at every budget on the three factorial machines (Fig. 3). The usual ways of spending

TABLE IV

THE DEPLOYED CACHE’S GAP TO EQ. (1), SPLIT BY ORACLES IN THE ENGINE (FIG. 2): MEAN SHARE OF THE GAP, IN PERCENT, WITH 95% t -INTERVALS OVER MACHINES.

Part of the gap	gpt-oss 11%			gpt-oss 25%		
	Panel (15)	New (5)	CPU only (3)	Panel (15)	New (4)	CPU only (3)
<i>Measured</i>						
What is cached: MIN’s set, read twice (MIN-2R)*	0 [−2, 3]	3 [−1, 8]	20 [10, 29]	21 [19, 22]	20 [18, 21]	37 [20, 54]
How it is read: the deployed set, read once (Dep-1R)	0 [−2, 2]	0 [−2, 1]	10 [−2, 22]	−3 [−7, 1]	−2 [−3, −1]	7 [−13, 28]
Both: MIN’s set, read once (MIN-1R)	33 [21, 44]	35 [22, 48]	44 [−3, 92]	31 [23, 39]	27 [23, 32]	45 [5, 84]
Then the read-ahead oracle	15 [11, 19]	19 [14, 24]	14 [5, 22]	21 [18, 23]	24 [21, 27]	21 [16, 26]
Left after all three	52 [43, 62]	46 [38, 55]	39 [13, 65]	48 [41, 55]	48 [44, 53]	37 [11, 63]
The in-step fetches themselves†	−	−	3 [−10, 16]	−	−	−3 [−17, 12]
<i>Attributed: of what is left</i>						
the GPU’s non-expert time (T_{GPU})	27 [25, 30]	28 [20, 36]	22 [12, 31]	34 [31, 37]	38 [33, 43]	29 [15, 43]
the oracle’s reads beyond MIN’s, at B_{host}	21 [20, 23]	18 [15, 20]	21 [12, 29]	19 [18, 21]	16 [15, 17]	20 [12, 29]
residual	4 [−6, 13]	1 [−3, 4]	−4 [−29, 21]	−5 [−12, 2]	−5 [−7, −3]	−12 [−32, 8]

Panel: the 15 machines of jobs 096–101 that ran every state. *New*: the 5 machines rented for the registered test (population and predictions fixed before any started; the deadline cut one 25% round). *CPU only*: the machines of the second registered test (Table V), whose arms ran with the in-step fetches off. *On the deployed path (Panel, New) the in-step fetches displace MIN’s set, so this row measures MIN’s schedule with the fetch table, not the set held. †The deployed cache with its fetch table against without it. *Both, then the read-ahead oracle* and *left* sum to the gap (in the CPU-only column, with †). The last three rows split *left* by attribution: T_{GPU} (2.9 ms) and the read-ahead oracle’s reads beyond R^* at B_{host} ; the residual is what they leave (Appendix L gives the probe sensitivity).

TABLE V
THE 2×2 ON BOTH READ PATHS, REGISTERED (GPT-OSS 11%).

Machine	Ratio	CPU only vs table	MIN-2R		Dep-1R		MIN-1R	Interaction (ms)
			fetch table	CPU only	fetch table	CPU only		
Ryzen 9 5950X‡	0.79	1.00±.00	1.12±.01	1.12±.01	1.06±.01	1.07±.00	1.39±.02	3.2 [3.0, 3.3]
Core Ultra 7 265K	0.57	1.06±.01	1.00±.01	1.10±.01	1.01±.00	1.03±.00	1.16±.01	0.4 [0.3, 0.5]
Ryzen 9 5950X	0.78	1.00±.01	1.01±.01	1.12±.01	1.01±.00	1.07±.01	1.39±.01	3.1 [2.9, 3.3]

Speeds relative to the deployed cache on the same machine and read path (*fetch table*: deployed; *CPU only*: in-step fetches off); MIN-1R ignores the table and is given against the CPU-only deployed cache. Subscripts: half-widths of 95% intervals over problems. Interaction: MIN-2R’s plus Dep-1R’s time minus the deployed cache’s and MIN-1R’s (CPU only, ms per token, 95% interval). ‡An all-zero fetch table: no in-step fetches on either path.

foresight waste reads. Prefetching without bypass, Belady’s original rule, admits experts MIN would bypass. Serving a miss on the CPU before copying it reads every admission twice. At the smallest budget of each model these gain at most 15% or lose, a trade-off known for disks [22] and CPU caches [23]. These gains were registered as bands per budget on two machines and held at most budgets. With the in-step fetches off, MIN’s set gains read twice as well, on the three machines tested (Section IV).

Admit as rarely as optimality allows: The greedy schedule admits whenever MIN allows and may evict an expert again before its next use. The fewest-admission schedule makes 0.60–0.73 of the greedy one’s copies for the same hits in replay. Read once in the step at gpt-oss 11% (Few-1R), it beat the deployed cache on all 15 machines that ran it stably, by a geometric mean of 1.26× (registered on 7 of them; two are RTX 4090s and two have server processors). Of the four machines that ran it unsteadily or failed a validity check, it lost on two, the two with the slowest links (ratios 0.14 and 0.21; Appendix I), and gained on the others.

VI. WITHOUT FORESIGHT, THE ENGINE RECOVERS LITTLE

The engine has two mechanisms that act on the same levers without foresight. A larger admission margin admits less: it cuts the deployed cache’s host reads by 4–6% at 11%. A layer-ahead copy applies the next layer’s router to this layer’s input and copies the top predicted expert ahead, so that a correctly predicted admission is read once. Four variants combine the two (Table II). We registered that none would run faster than 1.10× the deployed cache, or capture more than 35% of the read-ahead oracle’s gain, on any machine.

At 11% the best variant runs 1.007–1.024× the deployed cache, at most 6% of the read-ahead oracle’s gain. The counters say why: the variants read about as many host experts per token as the deployed cache (1.57–1.76 R^*). A prediction one layer ahead moves reads earlier but does not avoid them. On the slow-link RTX 4090s of Section VII, the variants with the layer-ahead copy lose time (0.64–0.72× at 11%), as registered, and the larger margin alone is the best (up to 26% of a smaller oracle gain).

VII. THE LINK-TO-CPU RATIO ON A SECOND CARD

On the RTX 5090 machines, the gain of every configuration that moves reads from the CPU to the link rises with the link-

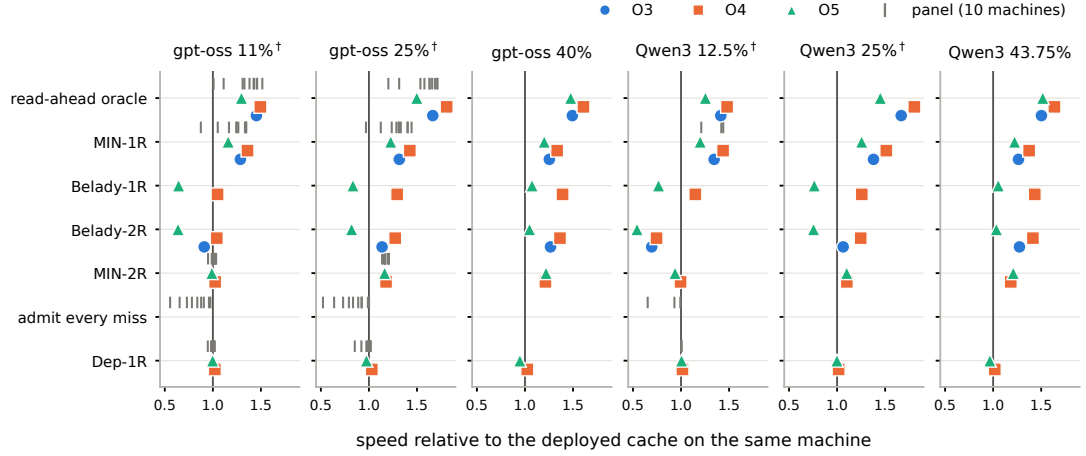


Fig. 3. What foresight is worth, by how it is spent: speed of each configuration of Table II relative to the deployed cache on the same machine, at the six budgets ([†]host-bound). Three machines at every budget (O3–O5: two Ryzen 9 9950X and a 9950X3D); the 10 machines of the panel’s main job (grey ticks) at the two host-bound gpt-oss budgets, and 3 of them at Qwen3 12.5%. The 95% intervals are mostly narrower than the markers. MIN-1R gains at every budget on these machines; reading twice or admitting what MIN bypasses gains at some budgets and loses at others.

TABLE VI

THE REGISTERED TESTS ON MACHINES RENTED FOR THEM: SPEED RELATIVE TO THE DEPLOYED CACHE ON THE SAME MACHINE (GEOMETRIC MEAN OF ITS ROUNDS) OF MIN-1R, THE READ-AHEAD ORACLE (*ahead*), AND THE BEST OF THE FOUR VARIANTS WITHOUT FORESIGHT (*none*: MARGIN, LA, LA-1R, LA-1R-MARGIN; SECTION VI), AND THAT VARIANT’S CAPTURE, ITS SHARE OF THE READ-AHEAD ORACLE’S GAIN.

Machine	Ratio	gpt-oss 11%				gpt-oss 25%			
		MIN-1R	Ahead	None	Capture	MIN-1R	Ahead	None	Capture
RTX 5090, rented for the test of policies without foresight									
Core Ultra 9 285K	0.54	1.22 $\pm$.01	1.46 $\pm$.01	1.02 $\pm$.00	4 $\pm$ 0%	1.24 $\pm$.01	1.64 $\pm$.02	1.03 $\pm$.00	4 $\pm$ 0%
Core i9-13900KF	0.59	1.25 $\pm$.01	1.46 $\pm$.01	1.01 $\pm$.00	2 $\pm$ 0%	1.26 $\pm$.01	1.61 $\pm$.02	1.01 $\pm$.01	2 $\pm$ 1%
Ryzen 9 5950X	0.84	1.37 $\pm$.01	1.50 $\pm$.01	1.01 $\pm$.00	2 $\pm$ 0%	—	—	—	—
Ryzen 7 9800X3D	0.88	1.15 $\pm$.01	1.34 $\pm$.01	1.01 $\pm$.01	4 $\pm$ 2%	1.22 $\pm$.01	1.55 $\pm$.02	1.01 $\pm$.00	2 $\pm$ 1%
Ryzen 9 7945HX	0.89	1.24 $\pm$.01	1.42 $\pm$.01	1.02 $\pm$.01	6 $\pm$ 1%	1.28 $\pm$.01	1.63 $\pm$.02	0.99 $\pm$.01	−1 $\pm$ 1%
RTX 4090, the second-card test									
Core i9-14900KF	0.37	0.96 $\pm$.00 [0.99]	1.13 $\pm$.01 [1.11]	1.03 $\pm$.00	26 $\pm$ 3%	1.02 $\pm$.00 [1.07]	1.30 $\pm$.01 [1.29]	1.04 $\pm$.00	13 $\pm$ 1%
Core Ultra 9 285K	0.39	1.01 $\pm$.00 [1.01]	1.20 $\pm$.01 [1.13]	1.04 $\pm$.00	20 $\pm$ 2%	1.06 $\pm$.01 [1.09]	1.36 $\pm$.01 [1.31]	1.04 $\pm$.00	12 $\pm$ 1%
Core Ultra 9 285K	0.40	1.01 $\pm$.00 [1.01]	1.20 $\pm$.01 [1.13]	1.04 $\pm$.00	20 $\pm$ 2%	1.06 $\pm$.01 [1.09]	1.37 $\pm$.01 [1.32]	1.05 $\pm$.00	12 $\pm$ 1%
RTX 4090, the second-card test at higher ratios									
Ryzen 7 7800X3D	0.57	1.13 $\pm$.01 [1.14]	1.30 $\pm$.01 [1.26]	1.01 $\pm$.01	4 $\pm$ 2%	1.17 $\pm$.01 [1.21]	1.48 $\pm$.01 [1.47]	1.02 $\pm$.00	5 $\pm$ 1%
Core i5-12400	0.77	1.31 $\pm$.01 [1.25]	1.46 $\pm$.01 [1.37]	1.01 $\pm$.00	3 $\pm$ 1%	1.35 $\pm$.01 [1.31]	1.69 $\pm$.02 [1.61]	1.02 $\pm$.00	3 $\pm$ 0%

Ratio: link-to-CPU. RTX 5090: job 109, whose population (desktop-class, link-to-CPU ratio at least 0.5) was fixed before any machine started. RTX 4090: job 111 (job 110’s predictions, committed before any RTX 4090 ran) and job 112 (the same predictions, on machines at higher ratios); in brackets, what the RTX 5090 machines’ trend in the link-to-CPU ratio predicted for that machine. Subscripts: half-width of the 95% interval over problems (paired bootstrap within the machine).

to-CPU ratio (Fig. 4; for MIN-1R, Spearman 0.89 over 19 machines). We froze that trend on the panel, one line in the log of the ratio per configuration, and registered that it would predict the oracles’ speed on RTX 4090 machines, a slower card behind a PCIe 4.0 link, within 0.10 in log.

It did, on all five RTX 4090 machines of two tests, at ratios 0.37–0.77: MIN-1R and the read-ahead oracle came within 7% of the prediction at 11% and 5% at 25% (Table VI); every machine’s 95% interval lies inside the band, and the interaction of Section IV is positive on the two with fast links, as registered. Predicting no change would have missed by up to 46% at 11% and 69% at 25%. The trend is coarser on fast links: applied after the fact to the new RTX 5090s of Section IV, it misses five of their 18 cells by more than the band, by up to 18%.

The ratio sets the direction of the gain more reliably than its size. Where the link reads at under a third of the CPU rate, Few-2R pays instead of MIN-1R (1.11–1.22 $\times$).

VIII. HOW MUCH FORESIGHT IS NEEDED

The oracles know the whole future; a predictor would know part of it, imperfectly. We give the cache a *window*, the routing of the next W tokens, exact or with each future expert right only with probability r . Throughout this section, the value of a window is its share of MIN’s gain over admitting every miss, the online policy that reads least at these budgets (Appendix G gives the policies).

Next-token prediction buys little: Exact routing of the next token recovers at most 0.31 of MIN’s gain at any host-bound

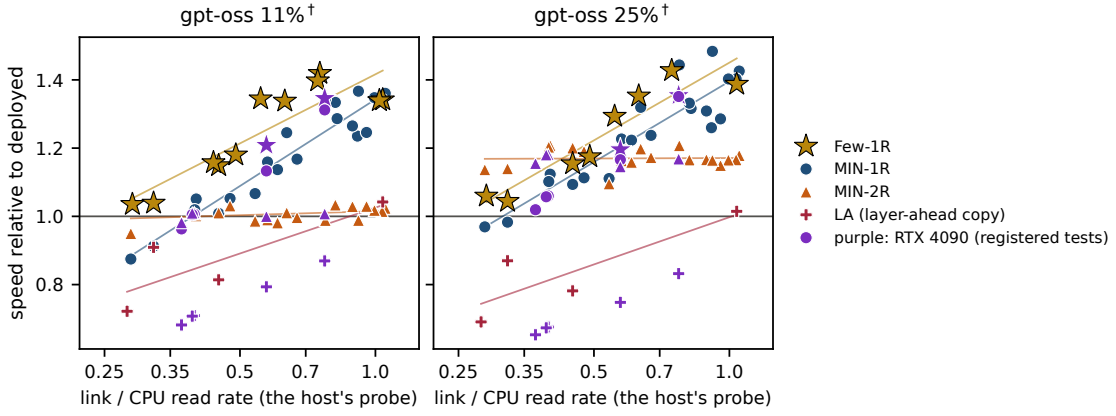


Fig. 4. The machine decides which way of spending foresight pays: speed relative to the deployed cache on the same machine against the machine’s link-to-CPU ratio, at the two host-bound gpt-oss budgets. Every marker is a machine that passed our checks. Stars: Few-1R. Circles and triangles: MIN-1R and MIN-2R, both on the greedy schedule. Crosses: the deployed cache with the layer-ahead copy. Purple markers: the RTX 4090 machines of the registered second-card test (Section VII), not used in the fits. Lines: least-squares fits in the log of the ratio, drawn for the trend only.

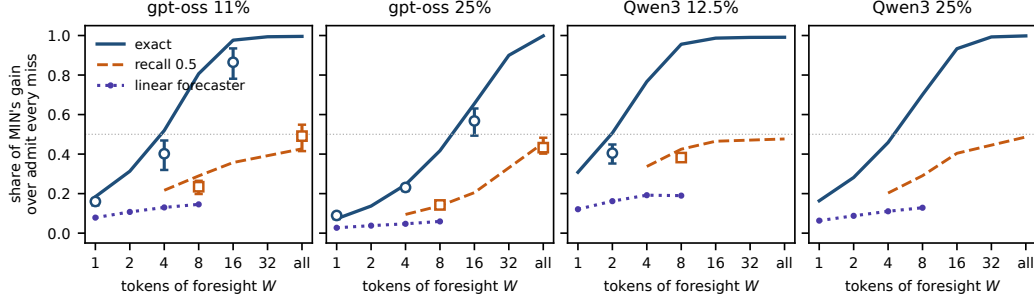


Fig. 5. Baseline here: *admit every miss*, not the deployed cache. The share of MIN’s gain over admitting every miss recovered, in host reads on the routing the engine runs, by exact routing of the next W tokens (solid), by a window whose experts are each right with probability 0.5 (dashed) and by a linear forecaster fitted on other text (dotted). Markers: the same share in time per token (mean and range over the panel’s main-job machines: 10 for gpt-oss, 3 for Qwen3 12.5%). Grey line: half of the gain.

budget of our two models; half of the gain at gpt-oss 11% needs 4 tokens (Fig. 5). In the engine, an exact 16-token window recovers 0.78–0.93 of the gain in time at 11% (registered at 0.80 or more; the lowest machine missed). Against the deployed cache, the same window copied in the step runs $0.89\text{--}1.32\times$ at 11% on 4 machines, below 1 on the 2 with the lowest link-to-CPU ratios. Accuracy matters as much as horizon: an 8-token window that is right half the time recovers 0.29 of the gain at 11%, about what 2 exact tokens buy (0.31).

The horizon is measured in distinct experts (exploratory): On 9 models’ own sampled text, the window that closes half of the read gap ranges from 0.6 to 34 tokens. Paging theory measures lookahead in distinct pages [24], [25], and so does the rule that fits: within that window a layer requests about $0.65C$ distinct experts ($0.66\text{--}0.81C$ on the AIME routing the engine runs), and it predicts a model’s horizon as well as a two-constant power law in tokens (Appendix F).

Nothing realisable reaches it: Forecasters trained on routing histories, linear or recurrent, close at most 15% of the read gap on gpt-oss: their precision falls with the horizon. Speculative batches save an online cache about a tenth of its reads at 90% draft acceptance and cost reads at 80% [26]–

[28] (Appendix F). Layer-ahead predictors [11], [12] reach 84% recall in our engine but see less than one token ahead (Section VI).

IX. WHAT THIS MEANS FOR SYSTEM BUILDERS

These rules come from one engine (ours), batch-1 decode and one prompt set, and rules 2–4 from one model at two budgets; elsewhere they are hypotheses to test, with Eq. (1) as the yardstick.

- 1) **Report the distance to the machine’s bound.** A speed-up over a chosen baseline hides how much is left. Equation (1) needs only the routing trace and one probe run.
- 2) **Make the read path keep the set the policy chose.** In-step fetches that put unchosen misses into slots undo a better set: with them, MIN’s set read twice closes none of the gap at the smallest budget in our engine; without them, on three machines at gpt-oss 11%, it closes 15–22% of the gap on its own and 22–56% when each admission is also read once.
- 3) **Do not expect a policy without foresight to get there.** Admitting less and predicting one layer ahead recover

at most 6% of what foresight is worth where it is worth most; the reads they leave are the ones only foresight avoids.

- 4) **Choose the read path by the link-to-CPU ratio.** Where the link is fast, copy admitted misses into their slots in the step; where it is slow, serve them on the CPU and copy in the background (Section VII).
- 5) **Issue reads before routing, and predict far enough ahead to save reads.** A read that waits for the router stays in series with the GPU’s non-expert time; only a prediction lets it start earlier, and only over the link (Eq. (3)). Predict the next $\approx 0.65C$ distinct experts of each layer, not the next token.

X. RELATED WORK

Offloading systems: Expert caches copy misses over PCIe [3]–[5], [29]; CPU-execution systems run non-resident experts on the host [6]–[8], [30]; FreeToken [9] and DALI [10] split each layer’s misses between the two. Predictors guess upcoming experts from the previous layer [11], [12], several tokens ahead [13], [31], from draft tokens [26]–[28], or decouple the router [32]. Each reports speed-ups over its own baseline; none bounds the machine (Appendix K covers many more).

Bounds and caching theory: WiSP [33], Budgeting Bytes [34], MoE-CAP [35] and the roofline [36] bound decode by bytes over bandwidth; Zhang [37] and Liang et al. [38] compare caches with MIN in hits. We add the time domain. MIN with bypass maximises hits [19], [20], not speed [18]; our fewest-admission schedule extends offline caching over reuse intervals [39].

XI. LIMITATIONS

- **Hardware, engine, sample and workload.** One engine, ours, for every engine result; one GPU model for most results and a second (RTX 4090) for two registered tests, all rented, mostly in desktop hosts; clocks could not be locked; batch-1 decode of two models on AIME prompts, also used while developing the system, teacher-forced on each model’s own greedy text; three tests deviated from their registration (Appendix D). The machines are a convenience sample of cheap offers, with few slow links.
- **The bounds.** Exact routing and per-layer slots; pooling slots across layers lowers MIN’s reads by 4.5–18.6%. The bounds drop latencies and are relative to one run of our probe, mostly taken while the model downloaded: the engine’s reads imply a rate more than 5% above the probe’s highest reading on 12 of 84 launch-budgets (Table S23). A second probe on the idle machine came within 4.9% of the first on 4 machines, but between two rentals of one machine the ratio moved by up to 12%. T_{GPU} is our engine’s kernel time on an RTX 5090.
- **The decomposition and the oracles.** The parts are measured at two budgets of one model; what the oracles leave is attributed, not measured. Oracles read a recorded

trace. Without the in-step fetches, the 2×2 ran on three machines.

- **Outputs and more.** Parity is measured by teacher-forced KL divergence against an all-VRAM reference (mean 0.0019 and 0.0005 nats), not by task accuracy; Appendix L gives the remaining caveats.

XII. CONCLUSION

MIN’s host reads at a machine’s probed rate bound batch-1 MoE decode from host memory; our cache reaches 31–54% of that bound. MIN’s set pays alone where the read path keeps it, more when read once; policies without foresight recover little.

REFERENCES

- [1] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler, “Data movement is all you need: A case study on optimizing transformers,” in *Proc. Mach. Learn. and Syst. (MLSys)*, 2021.
- [2] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, “Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks,” *J. Mach. Learn. Res.*, vol. 22, no. 241, 2021.
- [3] A. Eliseev and D. Mazur, “Fast inference of mixture-of-experts language models with offloading,” *arXiv preprint arXiv:2312.17238*, 2023.
- [4] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, “MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache,” *arXiv preprint arXiv:2401.14361*, 2024.
- [5] X. Song, Z. Zhong, R. Chen, and H. Chen, “ProMoE: Fast MoE-based LLM serving using proactive caching,” *arXiv preprint arXiv:2410.22134*, 2024.
- [6] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, “Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models,” *arXiv preprint arXiv:2402.07033*, 2024.
- [7] H. Chen, W. Xie, B. Zhang, J. Tang, J. Wang, J. Dong, S. Chen, Z. Yuan, C. Lin, C. Qiu, Y. Zhu, Q. Ou, J. Liao, X. Chen, Z. Ai, Y. Wu, and M. Zhang, “KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models,” in *Proc. 31st ACM Symp. Oper. Syst. Princ. (SOSP)*, 2025.
- [8] S. Zhong, Y. Sun, L. Liang, R. Wang, R. Huang, and M. Li, “HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference,” in *Des. Autom. Conf. (DAC)*, 2025, arXiv:2504.05897.
- [9] S. Yang, X. Fan, M. Pan, H. Xi, Z. Wang, S. Sun, K. Keutzer, S. Han, M. Zaharia, C. Xu, and I. Stoica, “FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution,” *arXiv preprint arXiv:2608.16157*, 2026.
- [10] Z. Zhu, G. Li, P. Wang, Z. Mo, M. Pei, Z. Song, X. Liang, and J. Cheng, “DALI: A workload-aware offloading framework for efficient MoE inference on local PCs,” *arXiv preprint arXiv:2602.03495*, 2026.
- [11] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, “Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference,” in *Proc. 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, 2024, arXiv:2308.12066.
- [12] S. Zhu, S. Bohl, R. Oester, and G. Alonso, “Pre-attention expert prediction and prefetching for mixture-of-experts large language models,” *arXiv preprint arXiv:2511.10676*, 2025.
- [13] Z. Wang, Y. Wang, L. Gong, C. Tang, W. Lou, T. Wang, C. Wang, and X. Zhou, “SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading,” *arXiv preprint arXiv:2609.12978*, 2026.
- [14] OpenAI, “gpt-oss-120b & gpt-oss-20b model card,” *arXiv preprint arXiv:2508.10925*, 2025.
- [15] A. Yang et al., “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [16] G. Gerganov and contributors, “llama.cpp: LLM inference in C/C++,” <https://github.com/ggml-org/llama.cpp>, 2023–2026.
- [17] L. A. Belady, “A study of replacement algorithms for a virtual-storage computer,” *IBM Syst. J.*, vol. 5, no. 2, pp. 78–101, 1966.
- [18] L. Zhang, R. Karimi, I. Ahmad, and Y. Vigfusson, “Optimal data placement for heterogeneous cache, memory, and storage systems,” *Proc. ACM Meas. and Anal. of Comput. Syst. (POMACS)*, vol. 4, no. 1, pp. 6:1–6:27, 2020.

- [19] M. C. Carlisle and E. L. Lloyd, "On the k -coloring of intervals," *Discrete Appl. Math.*, vol. 59, 1995.
- [20] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in *Proc. 43rd Int. Symp. Comput. Archit. (ISCA)*, 2016.
- [21] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Int. Symp. Memory Manage. (ISMM)*, 2013.
- [22] P. Cao, E. W. Felten, A. R. Karlin, and K. Li, "A study of integrated prefetching and caching strategies," in *Proc. ACM SIGMETRICS*, 1995.
- [23] A. Jain and C. Lin, "Rethinking Belady's algorithm to accommodate prefetching," in *Int. Symp. Comput. Archit. (ISCA)*, 2018.
- [24] S. Albers, "On the influence of lookahead in competitive paging algorithms," *Algorithmica*, vol. 18, no. 3, pp. 283–305, 1997.
- [25] D. Breslauer, "On competitive on-line paging with lookahead," BRICS, Tech. Rep., 1995.
- [26] L. Chen, Z. Wen, T. Wu, X. Zhang, and C. Wu, "SP-MoE: Speculative decoding and prefetching for accelerating MoE-based model inference," *arXiv preprint arXiv:2510.10302*, 2025.
- [27] S. Li, J. Lin, D. Ge, and Y. Ye, "MoE-SpAc: Efficient MoE inference based on speculative activation utility in heterogeneous edge scenarios," *arXiv preprint arXiv:2603.09983*, 2026.
- [28] A. Saxena, P.-A. Tsai, H. Taneja, A. Jaleel, and M. Qureshi, "Cascade: Utility-driven speculative decoding for mixture-of-experts," in *Proc. Mach. Learn. and Syst. (MLSys)*, 2026, arXiv:2506.20675 (2025).
- [29] P. Tang, J. Liu, X. Hou, Y. Pu, J. Wang, P.-A. Heng, C. Li, and M. Guo, "HOBbit: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433*, 2024.
- [30] Y. Zhang, S. Aggarwal, and T. Mitra, "DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference," in *Des., Autom. & Test in Eur. Conf. (DATE)*, 2025, pp. 1–7, arXiv:2501.10375.
- [31] S. Yadav and B. Asgari, "Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging," *arXiv preprint arXiv:2609.38090*, 2026.
- [32] R. Cai, Y. Ro, G.-W. Kim, P. Wang, B. Ehteshami Bejnordi, A. Akella, and Z. Wang, "Read-ME: Refactorizing LLMs as router-decoupled mixture of experts with system co-design," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 116 126–116 148, arXiv:2410.19123.
- [33] J. Zhang, L. Wu, M. Darbari, and L. Hong, "WiSP: A working-set view of mixture-of-experts serving on extremely low-resource hardware," *arXiv preprint arXiv:2606.21868*, 2026.
- [34] H. Zhang, "Budgeting bytes: A windowed storage roofline and dual-budget architecture ablations for storage-bound LLM decoding," *arXiv preprint arXiv:2609.04238*, 2026.
- [35] Y. Jiang *et al.*, "MoE-CAP: Cost-accuracy-performance benchmarking for mixture-of-experts systems," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025, arXiv:2412.07067.
- [36] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [37] Y. Zhang, "Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes," *arXiv preprint arXiv:2608.07911*, 2026.
- [38] J. Liang, S. Wang, M. Tian, Y. Li, D. Tang, and Z. Wei, "Not all models suit expert offloading: On local routing consistency of mixture-of-expert models," in *Int. Conf. Learn. Represent. (ICLR)*, 2026, arXiv:2505.16056.
- [39] D. S. Berger, N. Beckmann, and M. Harchol-Balter, "Practical bounds on optimal caching with variable object sizes," *Proc. ACM Meas. and Anal. of Comput. Syst. (SIGMETRICS)*, vol. 2, no. 2, 2018.